반도체장비데이터관리 08

SemiconDB 확장하기

김 민 진 | muea.mj@gmail.com

1) MaintenanceLog

왜 새 테이블이 필요한가

Equipment(equipment_id, model_name, install_date, status)

equipment_id	model_name	install_date	status
101	ETCH-A100	2021-05-10	active
102	CMP-X200	2023-11-23	maintenance
103	LITHO-Z300	2022-07-15	active
104	CVD-B500	2019-03-12	retired
105	DIFF-Y700	2023-01-20	active

- 장비의 상태가 maintenance라고만 적혀 있으면 충분한가?
- 언제 점검했는지, 누가 점검했는지, 무엇을 했는지는 어떻게 기록할까?

새로 추가할 테이블: MaintenanceLog

❖MaintenanceLog

- 장비의 유지보수 이력을 저장하는 테이블

컬럼명	타입	의미
maintenance_id	INT	유지보수 기록 고유 ID
equipment_id	INT	유지보수 대상 장비 ID
maintenance_date	DATE	유지보수 수행 날짜
engineer_name	VARCHAR(50)	유지보수 담당 엔지니어 이름
maintenance_type	VARCHAR(30)	유지보수 유형
details	VARCHAR(255)	작업 상세 내용

새로 추가할 테이블: MaintenanceLog

❖이 테이블은 이렇게 읽어야 한다:

- "누가(Engineer)가 어떤 장비(Equipment)를 언제(Date), 어떤 이유로(Type), 어떻게 처리했는가"

maintenance_id	equipment_id	maintenance_date	engineer_name	maintenance_type	details
1	101	2024-03-01	김서윤	정기점검	필터 교체 및 센서 점검
2	102	2024-03-02	이준호	고장수리	압력 이상으로 밸브 교체
3	101	2024-03-05	김서윤	정기점검	가스 라인 점검
4	103	2024-03-07	박민수	긴급수리	장비 멈춤 현상 해결
5	104	2024-03-10	최유진	정기점검	온도 센서 보정
6	102	2024-03-12	이준호	정기점검	내부 청소 및 상태 점검

예시 해석

- 1번 행: 김서윤이 101번 장비를 2024-03-01에 정기점검했고 필터 교체 및 센서 점검을 수행했다
- 2번 행: 이준호가102번 장비를 고장수리했다

새로 추가할 테이블: MaintenanceLog

- 질문
  - 4번 행을 문장으로 설명해보자.
  - 장비 101번은 몇 번 점검되었나?
  - 김서윤 엔지니어는 몇 번 작업했나?
  - maintenance_type은 몇 종류인가?

새로 추가할 테이블: MaintenanceLog

- 기존 테이블과의 관계

```
Equipment (1) : (N) UsageLog
```

```
Equipment (1) : (N) MaintenanceLog
```

- 장비 1대는 여러 번 사용될 수 있고, 여러 번 점검될 수도 있다.
  - UsageLog는 “누가 장비를 사용했는가”
  - MaintenanceLog는 “장비가 어떻게 관리되었는가”

CREATE TABLE

```sql
CREATE TABLE MaintenanceLog (
    maintenance_id INT PRIMARY KEY,
    equipment_id INT NOT NULL,
    maintenance_date DATE NOT NULL,
    engineer_name VARCHAR(50) NOT NULL,
    maintenance_type VARCHAR(30) NOT NULL,
    details VARCHAR(255),
    FOREIGN KEY (equipment_id) REFERENCES Equipment(equipment_id)
);
```

CREATE TABLE

- PRIMARY KEY : 유지보수 이력 한 건을 구분하는 고유 번호

```
maintenance_id INT PRIMARY KEY
```

- FOREIGN KEY : 유지보수 기록은 반드시 존재하는 장비에 대해서만 작성 가능

```
FOREIGN KEY (equipment_id) REFERENCES Equipment(equipment_id)
```

- NOT NULL : 비어 있으면 안 되는 핵심 정보 → 점검일, 담당자, 점검유형

```
maintenance_date DATE NOT NULL
engineer_name VARCHAR(50) NOT NULL
maintenance_type VARCHAR(30) NOT NULL
```

INSERT: 실습용 데이터 삽입

```sql
INSERT INTO MaintenanceLog
VALUES (1, 101, '2024-03-03', '최정우', '정기점검', '챔버 내부 청소 및 압력 점검');

INSERT INTO MaintenanceLog
VALUES (2, 102, '2024-03-06', '박수현', '수리', '연마 균일도 문제로 부품 교체');

INSERT INTO MaintenanceLog
VALUES (3, 104, '2024-03-09', '이도현', '정기점검', '노후 부품 상태 확인');

INSERT INTO MaintenanceLog
VALUES (4, 106, '2024-03-21', '김서윤', '초기점검', '신규 장비 설치 후 초기 테스트');
```

- 존재하지 않는 장비 참조 시 오류 발생 가능성 있음

```sql
INSERT INTO MaintenanceLog
VALUES (5, 999, '2024-03-25', '홍길동', '정기점검', '존재하지 않는 장비');
```

예시 쿼리

- 문제 1
  - 장비 101번의 유지보수 기록을 조회하세요.

```sql
SELECT *
FROM MaintenanceLog
WHERE equipment_id = 101;
```

- 문제 2
  - 이도현 엔지니어가 담당한 유지보수 기록을 조회하세요.

```sql
SELECT *
FROM MaintenanceLog
WHERE engineer_name = '이도현';
```

예시 쿼리

- 문제 3
  - 각 장비별 유지보수 횟수를 구하세요.

```sql
SELECT e.model_name, COUNT(*) AS maintenance_count
FROM MaintenanceLog AS m
JOIN Equipment AS e
  ON m.equipment_id = e.equipment_id
GROUP BY e.model_name;
```

- 문제 4
  - 유지보수가 한 번도 수행되지 않은 장비를 조회하세요.

```sql
SELECT e.model_name
FROM Equipment e
LEFT JOIN MaintenanceLog m
  ON e.equipment_id = m.equipment_id
WHERE m.maintenance_id IS NULL;
```

2) Department

1. 기존 상황 : EquipmentUser

- 현재 EquipmentUser 테이블 구조:

```sql
CREATE TABLE EquipmentUser (
    user_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    department VARCHAR(50),
    contact VARCHAR(20)
);
```

user_id	name	department	contact
1	김지훈	품질팀	010-1234-5678
2	박서윤	제조팀	010-2345-6789
3	이준호	품질팀	010-3456-7890
4	김민아	개발팀	010-4567-8901
5	정예린	연구팀	010-5678-9012
6	김지후	생산기술팀	010-6789-1111
7	한소라	품질팀	010-7890-2222

- 이 상태로 데이터를 계속 쌓으면 문제가 발생함!!

2. 문제점

- 문제 1. 부서명이 반복 저장됨
  - 예를 들어 사용자가 10명이고 그 중 6명이 “생산기술팀”이라면, department에 "생산기술팀"이라는 문자열이 6번 반복됨

  - 같은 정보가 여러 행에 중복 저장됨
  - 저장 공간 낭비
  - 관리 포인트 증가

2. 문제점

- 문제 2. 오타나 표기 불일치가 생길 수 있음
  - 예를 들어 아래처럼 들어갈 수 있습니다.
    - 생산기술팀
    - 생산 기술팀
    - 생산기술
    - 생산기술1팀
    - 이 경우, 사실상 같은 부서인데도 DB는 서로 다른 값으로 봅니다.

  - 그 결과:
    - 부서별 집계가 틀어짐
    - 검색 결과가 누락됨
    - 데이터 일관성이 깨짐

2. 문제점

- 문제 3. 부서 정보가 늘어나면 감당이 안 됨
  - 처음엔 부서명만 있으면 되지만, 나중에는 이런 정보가 필요할 수 있습니다.
    - 부서장
    - 사무실 위치
    - 연락처
    - 설명
  - 그런데 이걸 EquipmentUser에 넣으면, 사용자마다 부서 정보가 반복 저장됩니다.
  - 즉:
    - 한 부서의 부서장을 바꾸려면 여러 행 수정
    - 위치를 바꾸려면 여러 행 수정

2. 문제점 총 정리

❖문자열로 부서명을 저장하면 다음과 같은 문제가 생길 수 있다.

- 같은 부서명이 여러 번 반복 저장된다.
- 오타나 표기 차이로 데이터가 달라질 수 있다.
- 부서 정보를 별도로 관리하기 어렵다.
- 부서명이 바뀌면 여러 행을 수정해야 한다.

❖ *해결 방법*

→ *부서 정보를 별도 테이블로 분리하고, 직원 테이블은 부서 번호를 참조하도록 한다.*

3. Department 테이블

- Department 테이블은 부서 자체의 정보를 저장
  - EquipmentUser 테이블은 부서명을 직접 저장하는 대신 **부서 번호(dept_id)를 참조**하도록 할 것임

- Department 테이블 스키마

컬럼명	데이터 타입	의미	제약조건
dept_id	INT	부서의 고유 번호	PRIMARY KEY
dept_name	VARCHAR(50)	부서명	NOT NULL, UNIQUE
manager_name	VARCHAR(50)	부서장 이름	
office_location	VARCHAR(50)	부서 사무실 위치	

3. Department 테이블

- Department 테이블 데이터

dept_id	dept_name	manager_name	office_location
1	제조팀	김부장	A동 2층
2	품질팀	이과장	B동 1층
3	연구팀	박차장	A동 1층
4	개발팀	최팀장	연구동 3층
5	생산기술팀	송팀장	연구동 3층

Department 생성 & EquipmentUser 구조 변경

```sql
-- 1. Department 테이블 생성
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL UNIQUE,
    manager_name VARCHAR(50),
    office_location VARCHAR(50)
);

-- 2. Department 데이터 입력
INSERT INTO Department VALUES (1, '제조팀', '김부장', 'A동 2층');
INSERT INTO Department VALUES (2, '품질팀', '이과장', 'B동 1층');
INSERT INTO Department VALUES (3, '연구팀', '박차장', 'A동 1층');
INSERT INTO Department VALUES (4, '개발팀', '최팀장', '연구동 3층');
INSERT INTO Department VALUES (5, '생산기술팀', '송팀장', '연구동 3층');
```

Department 생성 & EquipmentUser 구조 변경

```sql
-- 3. EquipmentUser에 dept_id 컬럼 추가
ALTER TABLE EquipmentUser ADD dept_id INT;


-- 4. 기존 department 값을 기반으로 EquipmentUser의 dept_id 채우기
UPDATE EquipmentUser SET dept_id = 1 WHERE department = '제조팀';
UPDATE EquipmentUser SET dept_id = 2 WHERE department = '품질팀';
UPDATE EquipmentUser SET dept_id = 3 WHERE department = '연구팀';
UPDATE EquipmentUser SET dept_id = 4 WHERE department = '개발팀';
UPDATE EquipmentUser SET dept_id = 5 WHERE department = '생산기술팀';
```

Department 생성 & EquipmentUser 구조 변경

```sql
-- 5. EquipmentUser의 dept_id를 NOT NULL로 변경
ALTER TABLE EquipmentUser
MODIFY dept_id INT NOT NULL;

-- 6. EquipmentUser에 외래키 추가
ALTER TABLE EquipmentUser
ADD CONSTRAINT fk_employee_department
FOREIGN KEY (dept_id)
REFERENCES Department(dept_id);

-- 7. EquipmentUser의 기존 department 컬럼 삭제
ALTER TABLE EquipmentUser
DROP COLUMN department;
```

Department 생성 & EquipmentUser 구조 변경

```sql
-- 8. 최종 구조 확인
SELECT * FROM Department;

SELECT * FROM EquipmentUser;
DESC EquipmentUser;
```

user_id	name	contact	dept_id
1	김지훈	010-1234-5678	2
2	박서윤	010-2345-6789	1
3	이준호	010-3456-7890	2
4	김민아	010-4567-8901	4
5	정예린	010-5678-9012	3
6	김지후	010-6789-1111	5
7	한소라	010-7890-2222	2

↓ DESC EquipmentUser 결과화면

Field	Type	Null	Key	Default	Extra
user_id	int	NO	PRI	NULL	
name	varchar(20)	YES		NULL	
contact	varchar(50)	YES		NULL	
dept_id	int	NO	MUL	NULL	

핵심 정리!!

- 테이블 이름은 데이터의 의미를 반영해야 한다
- 데이터 구조는 고정된 것이 아니라 변화한다
- 문자열 데이터를 참조형 데이터로 바꾸면 구조가 더 안정적이 된다
- 기존 데이터를 유지하면서도 구조를 개선할 수 있다

끝!